Solution Code - Spring Data JPA with Hibernate - JPA_Unit_Testing

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: src\main\java\com\example\jpa\JpaApplication.java

```
 1 | package com.example.jpa;
 2 |
 3 | import org.springframework.boot.SpringApplication;
 4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
 5 | import org.springframework.data.jpa.repository.JpaRepository;
 6 | import jakarta.persistence.Entity;
 7 | import jakarta.persistence.Id;
 8 |
 9 | @Entity
10 | class Customer {
11 |     @Id
12 |     private Long id;
13 |     private String name;
14 | }
15 |
16 | interface CustomerRepository extends JpaRepository<Customer, Long> {}
17 |
18 | @SpringBootApplication
19 | public class JpaApplication {
20 |     public static void main(String[] args) {
21 |         SpringApplication.run(JpaApplication.class, args);
22 |     }
23 | }
```

Source File: src\test\java\com\example\jpa\JpaApplicationTest.java

```
 1 | package com.example.jpa;
 2 |
 3 | import org.junit.jupiter.api.Test;
 4 | import org.springframework.beans.factory.annotation.Autowired;
 5 | import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
 6 | import static org.assertj.core.api.Assertions.assertThat;
 7 |
 8 | @DataJpaTest
 9 | public class JpaApplicationTest {
10 |     @Autowired
11 |     private CustomerRepository repo;
12 |
13 |     @Test
14 |     public void testSaveCustomer() {
15 |         Customer c = new Customer("Alice", "alice@example.com");
16 |         Customer saved = repo.save(c);
17 |         assertThat(saved.getId()).isNotNull();
```

Solution Code - Spring Data JPA with Hibernate - JPA_Unit_Testing

```
18 |     }
19 | }
```